

C++ LEARNING PLAN · DAY 01

The Compile Model

C++ Starts Here

Your code is now something you build before you run.

Same syntax — new build step

This slide: what carries over from Python and Arduino, and the one genuinely new idea this week.

ALREADY YOURS

▶ Arduino C++ surface.

setup()/loop(), digitalWrite, if, basic types — you've written these.

▶ Strong Python OOP.

Vector3D, Shape, the Scene state machine. The concepts transfer.

▶ if / loops / functions.

Control flow looks almost identical to what you know.

NEW IN C++

▶ Compile, then run.

A separate build step turns your file into a program. Python skipped this.

▶ Static types.

You declare what each variable is, and the compiler enforces it.

▶ The toolchain.

clang++ on the command line, not a Run button.

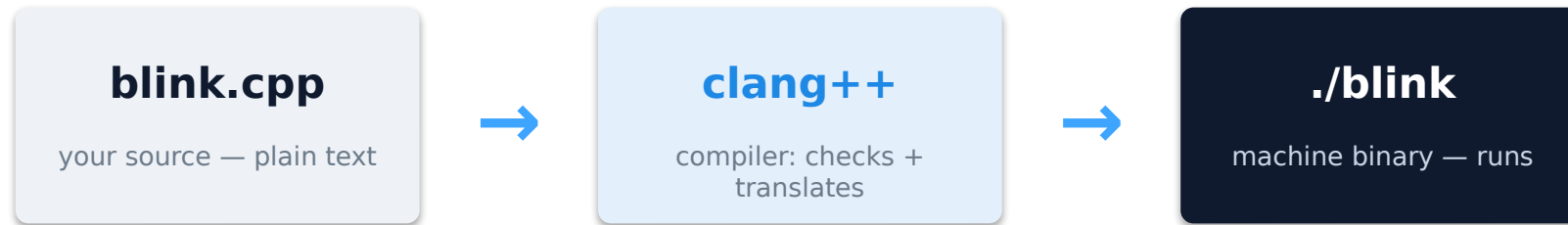


TAKEAWAY

Python runs your source directly. C++ turns it into a machine binary first — that single shift is the whole of today.

Compiled, not interpreted

This page: what 'compiling' means, and why C++ needs a build step that Python doesn't.



Python, by contrast, has no build step — the interpreter reads and runs your source line by line.

Python: one step
`python3 blink.py`

// C++: build, then run
`clang++ blink.cpp -o blink && ./blink`

► **Checked before it runs.**

Type mistakes are caught at compile time, not mid-execution.

► **Standalone & fast.**

The binary is machine code — no interpreter needed to run it.

One line builds it and runs it

This page: the exact command you'll type all week, taken apart flag by flag.

```
clang++ -std=c++17 -Wall blink.cpp -o blink && ./blink
```

clang++

the compiler — the program that builds your code

-Wall

show all warnings — catches mistakes the compiler would let slide

-o blink

name the output program 'blink' (without -o you get 'a.out')

-std=c++17

use the modern 2017 standard, so newer features work

blink.cpp

your input file

&& ./blink

if the build succeeded, run it; the && chains the two steps

TAKEAWAY

Edit the file, rerun this one line. The && means a broken build never runs stale code.

Every program needs main()

This page: the smallest complete C++ program, and what each line is doing.

```
#include <iostream>

int main() {
    std::cout << "Hello, C++\n";
    return 0;
}
```

OUTPUT

Hello, C++

▶ **#include <iostream>**

Pulls in the input/output library so cout exists.

▶ **int main()**

The entry point. Execution always starts here.

▶ **{ ... }**

The braces hold the function body — the steps to run.

▶ **return 0;**

Tells the OS it finished cleanly. 0 means success.

Printing and reading: cout and cin

This page: how C++ does print() and input() — using the << and >> operators instead.

```
int x;  
std::cout << "Enter a value: ";  
std::cin >> x;    // read into x  
std::cout << "x = " << x << "\n";
```

OUTPUT

```
Enter a value: 42  
x = 42
```

▶ **<< sends out.**

Pushes data to the screen. Chain several with more <<.

▶ **>> reads in.**

Pulls typed input into a variable.

▶ **std::**

The standard library's namespace. cout/cin live inside it.

▶ **"\n"**

Ends the line. std::endl does the same, slightly slower.

Where is main() on an Arduino?

This page: the IDE hides main() from you — here is roughly what it generates behind the scenes.

```
// what the Arduino IDE writes for you  
int main() {  
    setup();           // runs once  
    while (true) {  
        loop();        // runs forever  
    }  
}
```

► **It's still C++.**

Your setup() and loop() are ordinary C++ functions.

► **The IDE adds main().**

It calls setup() once, then loop() in an endless while.

► **Same language, less boilerplate.**

Desktop C++ just makes you write main() yourself.

 TAKEAWAY

Desktop C++ this week and your Arduino sketches are the same language — Week 4 just removes the IDE's training wheels.

SUMMARY

Python vs C++ vs Arduino

This page: the same four ideas across the three environments you'll move between.

Aspect	Python	C++ (desktop)	Arduino
Run model	interpret	compile, then run	compile + flash
Entry point	top of file	int main()	setup() + loop()
Print	print()	std::cout	Serial.print()
Types	dynamic	static	static

 TAKEAWAY

You already think in the Python column — this week makes you fluent in the C++ one.

TODAY'S EXERCISE · 30-45 MIN

Build `blink_console.cpp`

Recreate your Arduino blink on the desktop: print **LED ON** / **LED OFF** in a loop, with a pause between each, using only what's on these slides.

STRUCTURE

- › One .cpp file
- › `#include <iostream>`
- › `int main() { ... }`
- › `return 0;` at the end
- › Build with `-std=c++17 -Wall`

BEHAVIOR

- › A loop that runs ~5 times
- › Print LED ON, then LED OFF
- › `cout <<` with `\n` each line
- › A short pause between states

STRETCH

- › A real pause needs `<thread>`
- › plus `<chrono>`
- › `sleep_for(milliseconds(500))`
- › Verify the headers compile
- › Aim for a clean `-Wall` build

Things that bite on day one

Missing semicolon

Every statement ends in ; — the compiler often points at the NEXT line, so look one line up from the error.

Forgot to rebuild

Editing the .cpp doesn't change ./blink until you recompile. Rerun the full clang++ line — the && does it for you.

Read the FIRST error

One mistake cascades into a wall of errors. Fix the top one, recompile, and most of the rest vanish.

std:: is not optional

cout, cin, string all live in std. Forgetting the prefix is the most common beginner error.

Day 1 down — the toolchain works

DAY 2

Types & variables

Static typing in practice:
int / float / char / bool.
Why integer overflow matters
on a 2 KB microcontroller.

DAY 5

Headers & files

.h declares, .cpp defines.
Compile several files into
one program — the build
model from today, scaled up.

WEEK 4

Onto the Arduino

The same C++, flashed to
the Nano. main() comes back,
this time hidden by the IDE
you already met on slide 5.

You can compile and run C++. Every other day this week stands on that one loop.